STRATA: Layer-by-Layer Point-in-Time Checking of ML Pipelines Written by Coding Agents

Pavel Marian, Stanislav Chumakov, Danil Ezhov, Alexander Boukhanovsky
AI Institute, ITMO University, Saint Petersburg, Russian Federation
{pmarian, svchumakov}@itmo.ru

***Abstract*—We present STRATA, a point-in-time checker for ML pipelines that coding agents write over timestamped relational data. The tool runs every layer of a pipeline (task, features, model, in-context examples, agent) on the data as it is and as it was at the prediction time; any difference in the layer's output is a LEAK witness that names the time, entity, column and channel. A database enters only through an availability schema, which STRATA drafts from the raw files, with an optional LLM agent for what column names do not say. On eight benchmark databases STRATA matches all 34 control verdicts and kills 43 of 43 point-in-time mutants with no false alarm on 55 harmless ones. On 134 generated feature programs it finds 16 leaks that a row-cut oracle misses, and all 27 agent-written training runs on RelBench learn from rows after the prediction time. Demo: https://jrzkaminski-strata-demo.static.hf.space. Video: [TODO: video URL, e.g. https://youtu.be/...]**

***Index Terms*—data leakage, point-in-time correctness, ML pipelines, coding agents, differential testing**

I. Introduction

Successful execution of an ML pipeline written by a coding agent does not guarantee its temporal correctness. A pipeline that filters on one reference timestamp can still read information that did not exist at the prediction time, because availability is spread across tables, columns and the code that trains the model. Such violations yield plausible features and inflated offline results that do not reproduce at serving time [1], [2]. The property that every output computed for time t depends only on facts available at t is called point-in-time (PIT) correctness. Existing tools look at one place: static analysis of notebook code [3], learned detectors over code [4], distribution checks in pipelines [5], or single-feature AUC thresholds in AutoML products [6], [7]. Feature frameworks offer cutoff times [8] but trust the user to know when each value becomes known.

When an agent writes the whole pipeline, a leak can sit in any of five layers: the task (which entities and labels exist at t), the features, the model (which training rows it learns from), the in-context (ICL) examples, and the agent itself (what it recalls about known entities instead of reading the data). A check of one layer passes a pipeline that leaks in another: agent-written scripts that cut their features correctly still train on future labels (Section IV). Agents also meet databases nobody has annotated, so a checker tied to one schema leaves them without a gate.

STRATA closes this gap with one principle applied to every layer: a layer is PIT-correct at t if its output on the full database D equals its output on $D_{\leq t}$, the database as it was at t. It contributes: (i) a database-agnostic oracle: a database enters only through an availability schema of a few lines per table, which strata init drafts from the raw files, optionally with an LLM agent, for review; (ii) five black-box layer checks with a verdict vocabulary that keeps unknowns apart from passes and attaches a witness to every leak; (iii) agent integration: a CLI with exit codes and JSON reports, a sandbox for code written by a model, and a skill file that a coding agent loads on its own.

II. The STRATA Tool

Availability schema. The schema is the only input specific to a database, and every check is only as good as it. Each table is timed (a row exists from its time column on), inherits its time through a key, or is static. Late-filled values are declared as self-available (a delivery date is known at its own value) or with a stamp of their own (value_time: a rating is known at rated_at). A stamp may also be a fixed date or null: a value stored as its state at the end of the data (a final status, a current price, a career total) is then never known before that date. Mutable columns whose validity cannot be determined are unknown. strata init drafts the schema from CSV or Parquet files: the earliest date column becomes the row time, later date columns become self-available, tables without dates inherit time through a shared key, and text columns named like a status become unknown. Names and dates do not reveal which values are snapshots taken at the end of the data, so strata init --llm adds a schema agent. An LLM (through OpenRouter) reads a profile of the files, the heuristic draft and optional documentation, and returns every decision with a reason in a fixed JSON schema; STRATA checks each decision against the data (missing columns, unparsable or late row times, broken inheritance chains) and returns the findings for at most two revisions. The model never produces a verdict or code.

Database states. For any t, $D_{\leq t}$ removes rows that did not exist and blanks values not yet filled. Some channels survive truncation, so the canary D' keeps $D_{\leq t}$ intact but rewrites everything $D_{\leq t}$ removes with marker values (shifted numbers, dates beyond the data horizon, a reserved category).

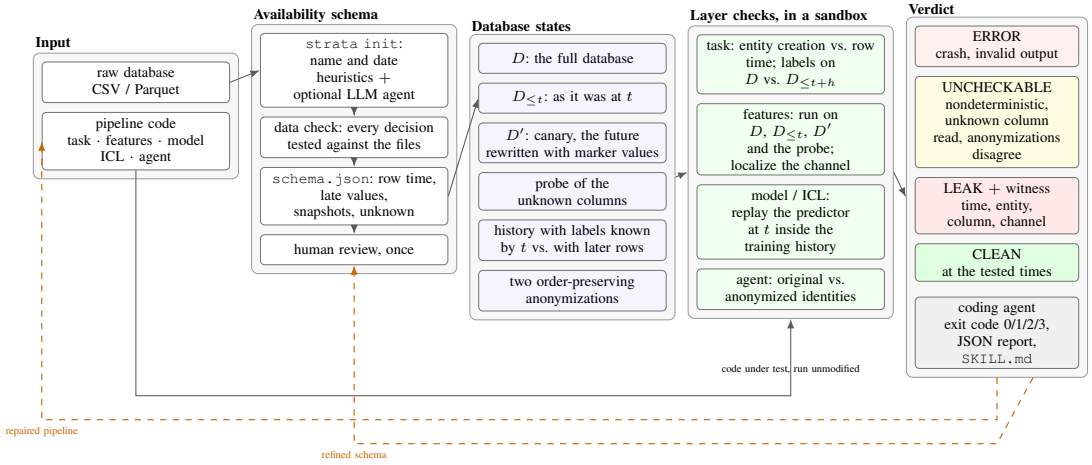


Fig. 1. STRATA workflow: the schema drafted from the raw files and reviewed once, the database states derived from it, the layer checks that run the code unmodified in a sandbox, and the verdict that goes back to the agent.

The unknown probe perturbs only unknown columns; keys are never perturbed.

Layer checks (Fig. 1). The lower bound of the label check targets windows closed at t: events stamped exactly at t move a microsecond into the past, so history is unchanged but such a window loses them. Any pipeline function $f(db, ids, t)$, including a training routine that builds its own examples, is checked like features, and a feature leak is localized by cutting one channel at a time (rows of one table, or one late-filled column) until the output changes. The model and ICL checks replay a predictor at a query time inside the training history; a label counts as known at t when its row time plus the label horizon h is at most t. The agent check needs two anonymizations, not one. Both preserve order and equality, so a model that learns only from the data cannot tell them from the original identities: real identities that change the output signal recall or an external lookup, while anonymizations that disagree with each other reveal a dependence on how identities are spelled (hashed ID features) and yield UNCHECKABLE instead of a false alarm.

Verdicts. The verification is one-sided: LEAK is a finite witness of a violation, CLEAN only says that none was observed at the times tested, and neither UNCHECKABLE nor ERROR counts as clean. Each predictor is first replayed twice on identical input to detect nondeterminism.

Agent integration. Every check is one command with exit code 0/1/2/3 for CLEAN/LEAK/UNCHECKABLE/ERROR and a JSON report whose witness names the time, entity, column and channel. Code under test runs in a forked child with a timeout, a memory limit and an environment stripped of API keys. A SKILL.md file says when to call which check, how to read a witness, that only a rerun may declare a leak fixed, and that CLEAN is not a proof. The checker is about 1,300 lines of Python on pandas with 38 tests; the examples for all ten databases differ only in schema files and command templates.

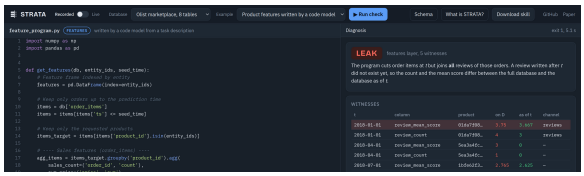


Fig. 2. The demo workbench on a recorded run: the program under test, the verdict and its witnesses (time, product, both values, channel).

III. Demonstration

The demo is a browser workbench over recorded STRATA runs on real data (Fig. 2). A feature program written by a code model for Olist gets LEAK, localized to the reviews table, is repaired in place and rerun CLEAN; a program that reads order_status gets UNCHECKABLE with the output columns that depend on it. The schema viewer shows the drafted Olist schema with the LLM agent's reason per table. An agent-written RelBench [9] script leaks in the model check; a second script gets UNCHECKABLE in the identity check because its two anonymizations disagree. A second page runs STRATA itself in the browser on a sample of Olist: a model chosen by the viewer writes the program or drafts the schema from an editable prompt, and the checks run on the result.

IV. Evaluation

The experiments answer four questions, each backed by one table: does STRATA transfer across databases without code changes and separate violations from harmless changes (Table I); is its verdict more reliable than that of an oracle written for one database (Table II); can the schema be drafted without hand annotation (Table III); and what does it find in training code written by agents (Table IV).

Eight databases (PITBench). PITBench [10] brings eight relational databases to one contract: an availability map, 34 control programs with known verdicts, and a reference pipeline per database. Each map was translated into a STRATA

TABLE I

ONE CODE BASE AND ONE SCHEMA PER DATABASE: STRATA GETS EVERY CONTROL VERDICT AND SEPARATES VIOLATING FROM HARMLESS MUTANTS ON ALL EIGHT PITBENCH DATABASES, WHILE EACH WEAKER DETECTOR FAILS ON A DIFFERENT KIND OF LEAK. CONTROLS: LEAKING PROGRAMS DETECTED AND CORRECT PROGRAMS FLAGGED. MUTANTS: BEHAVIOUR-CHANGING MUTANTS, HAND-LABELLED VIOLATIONS KILLED, HARMLESS MUTANTS FLAGGED. LEAK CLASSES: R ROWS NOT CUT AT t, L LATE-FILLED VALUE, S SNAPSHOT, A DUMP-TIME AGGREGATE, B SAME-DAY BOUNDARY, E ENTITY BEFORE IT EXISTS, N NEIGHBOUR STATISTIC.

				Controls				Mutants of the reference pipeline				
				STRATA		rows	+ late		violations killed		harmless flagged	
Database	Domain	Rows	Classes	leaks	correct	cut	+ canary	live	STRATA	feat. only	STRATA	any change
Olist	marketplace	0.94M	R L S	3/3	0/2	1/3	2/3	10	5/5	1/5	0/5	5/5
F1	sports	59k	R B A	3/3	0/1	2/3	2/3	10	5/5	1/5	0/5	5/5
PKDD'99	banking	1.08M	R E L S S	5/5	0/1	2/5	2/5	15	6/6	1/6	0/9	9/9
AdventureWorks	retail	174k	R L S	3/3	0/1	1/3	2/3	13	5/5	1/5	0/8	8/8
Northwind	wholesale	3.2k	R L S	3/3	0/1	1/3	2/3	14	6/6	1/6	0/8	8/8
Sakila	rental	39k	R L S	3/3	0/1	1/3	2/3	10	5/5	1/5	0/5	5/5
MovieTweetings	recsys	1.12M	R N	2/2	0/1	2/2	2/2	10	5/5	1/5	0/5	5/5
TPC-H	order entry	0.43M	R L S	3/3	0/1	1/3	2/3	16	6/6	1/6	0/10	10/10
Total				25/25	0/9	11/25	16/25	98	43/43	8/43	0/55	55/55

schema mechanically (time columns, late-filled dates, snapshot columns as dated or null stamps), and the controls ran unmodified at six times each. Table I shows that a schema is all a new database needs: the verdicts do not depend on the domain, size or leak classes, and the localized channel is the one the control leaks through (rental.return_date, product.list_price). The schema, in turn, decides what can be seen. Declaring the snapshot columns unknown turns the 8 snapshot and aggregate controls into UNCHECKABLE and changes nothing else; a naive F1 schema that times results at midnight lets the leak at the race-day boundary pass as CLEAN. A median control takes 2.6 s.

Mutants. Five operators (remove a time cut, move a window past t, widen the label horizon, swap </<=, remove a lower bound) produce 144 mutants of the reference pipelines; the 98 that change some output were labelled by hand as violations or not. select_entities, get_features and fit_predict are checked like features, make_label by the label check. In the code and in the output, a harmless mutant (a longer window into the past, a stricter cut) looks like a violation; only the comparison with the database at t tells them apart, on every database (Table I). Most violations sit outside the feature code: the training routine fires in 32 kills and the label check in 19, and three kills need the probe of the lower bound.

Baselines (Table I). Each weaker detector fails on a different kind of leak, and the classes column shows which. Cutting rows at t, as cutoff times in feature frameworks do, cannot see a value filled in later or a snapshot, so it falls short exactly on the databases with classes L and S; adding late-filled dates and a canary, as an oracle for one database does, still misses every snapshot. The feature layer alone misses the violations in label windows and training routines. Treating any output change as a leak, as regression testing would, cannot tell a violation from a harmless change. A static rule that flags code naming a late column or never comparing with t, given the same schema, finds 22 leaking controls with 4 false alarms but only 6 violating mutants with 9 false alarms: a mutated comparison still looks like a cut. The AutoML rule, a feature with AUC above 0.85 (0.80), finds 4 (5) leaks and flags 3 of 9 correct programs: leaked features need not be predictive, and good features are.

TABLE II

A CLEAN FROM AN ORACLE WRITTEN FOR ONE DATABASE IS NOT RELIABLE: STRATA CONFIRMS THE ORACLE'S LEAKS, BUT HALF OF THE PROGRAMS THE ORACLE PASSES LEAK OR CANNOT BE CHECKED. ROWS: VERDICT OF A HANDWRITTEN ROW-CUT ORACLE ON 134 GENERATED PROGRAMS.

Row-cut oracle	LEAK	CLEAN	UNCH.	ERROR	Main channel
LEAK (63)	62	0	0	1	reviews rows
CLEAN (71)	16	35	20	0	delivery dates

Olist (features). Corpus: the Olist e-commerce database and one task, whether a product is ordered in the 90 days after t. Two code models wrote 250 feature programs from a task description that never mentions leakage; 134 execute. The reference is a handwritten oracle for Olist alone that cuts rows by table time; both checkers ran with the same entities and three prediction times. Table II shows that the two agree on the oracle's leaks and part on its clean verdicts. The one leak STRATA does not confirm is a program truncated by the model's output limit, which returns no frame. The programs the oracle wrongly passes filter orders by purchase time but compute delivery times from delivery dates (15) or the times of review answers (1) that were still in the future at t; a row cut cannot see a value filled in later. Declared self-available in the schema, these columns produce ordinary witnesses, so the canary adds no further program. The UNCHECKABLE programs read order_status, which Olist overwrites without history; no oracle can check them, and STRATA says so instead of passing them.

Schema drafting (Table III). Schemas for the eight PIT-Bench databases were drafted from the raw files alone, with the heuristics and with the agent (Claude Opus 5.5 without tool access, so the maps could not be read). Snapshots decide

TABLE III
DRAFTED SCHEMAS ON THE EIGHT PITBENCH DATABASES AGAINST THE HAND-MADE MAPS, AND THE 34 CONTROLS RUN ON THE DRAFTS. NAMES AND DATES SUFFICE EXCEPT FOR SNAPSHOTS, WHICH THE LLM AGENT FINDS. CAUGHT AND FLAGGED MEAN LEAK OR UNCHECKABLE.

	Heuristic	LLM agent
Row times as in the map	18/26	23/26
Late-filled dates found (extra)	12/14 (22)	13/14 (19)
Snapshot columns caught	5/50	50/50
Leaking controls caught	23/25	25/25
Verdict as expected	26/34	30/34
Correct controls flagged	3/9	3/9

TABLE IV
ALL 27 AGENT-WRITTEN TRAINING RUNS ON RELBENCH LEARN FROM ROWS AFTER t; THE QUERY AUC RISES IN ONLY 5–6 OF THE 9 RUNS PER TASK. AUC: MEDIAN, AND RUNS IN WHICH IT RISES.

	Rows			Query AUC			Identity	
Task	$\leq t$	$> t$	max $\lvert\Delta s\rvert$	at t	full	rises	CLEAN	UNCH.
f1/dnf	7,729	3,650	0.04–0.86	0.91	0.95	6	6	3
f1/top3	919	412	0.06–1.00	0.71	0.82	5	7	2
event/ignore	3,755	14,727	0.13–0.96	0.79	0.92	6	6	3

the outcome: on the heuristic drafts two F1 leaks (race-day boundary, career totals) pass as CLEAN, on the agent's drafts none does. The heuristics take a wrong date as the row time (the shipped date of Northwind orders, the date from which the PKDD'99 district statistics are known), and the missing rows look like a leak. The agent's flags are cautious or deliberate: it declares two mutable columns unknown (Northwind freight, Sakila payment amount), which gives UNCHECKABLE, and it times Olist reviews by the customer's answer, not by the survey date, since the score does not exist before the answer. It kept that decision with a reason when the data check queried it and listed it as a question for the data owner. A schema takes 1–3 calls, about a minute per database; two runs differ in one decision.

RelBench (model and agent). Three isolated sessions of one commercial coding agent each wrote three training scripts for three binary tasks (27 runs). STRATA runs them unmodified through one command template, with the official label horizons (30 days for F1, 7 for Event). Every run is LEAK: the scripts train on the whole table they are given and never bound it by the label horizon. The metric would not have revealed this: the median AUC rises on every task, but not in 10 of the 27 runs, so the verdict rests on the outputs. The identity check finds no recall; its UNCHECKABLE runs come from three scripts whose outputs differ between the two anonymizations (hashed or scaled ID features), and a comparison of original and anonymized identities alone would report them as recall.

Cost. On four cores a feature check of an Olist program takes a median 6.5 s, a model check of an agent script 5–6 s.

Limitations. A schema is a claim about the data, and a verdict is only as good as its schema; CLEAN is scoped to the tested times and entities; the model and ICL checks need a time column and a declared label horizon; the agent check detects identity dependence, not pretraining as such; one base model wrote the RelBench corpus; the PITBench controls and mutation labels have one annotator, and the agent and ICL checks have no natural positives yet, so zero false alarms is an observation, not a bound; the schema agent sends a profile with sample values to the model provider (--no-samples keeps them local).

V. RELATED WORK

Temporal leakage is a recurring source of irreproducible ML results [1], [2]. Static detectors find leakage patterns in notebooks [3], and learned models classify code [4]; both judge code, whereas STRATA judges outputs and therefore treats model training and context assembly like features. The verification is one-sided, as in look-ahead freedom as temporal non-interference [11], a static formulation for one layer of backtesting code. Featuretools cutoff times [8] prevent row leakage when availability is given; STRATA stamps values as well as rows.

VI. AVAILABILITY

Demo (project page, workbench, live run, usage instructions): https://jrzkaminski-strata-demo.static.hf.space. Code, tests, the schemas and replay scripts for Olist, RelBench and PITBench, run logs, the 27 agent-written scripts with replay inputs, and the skill file: [TODO: GitHub / Zenodo URL], [TODO: license].

REFERENCES

[1] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[2] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 2023.

[3] C. Yang, R. A. Brower-Sinning, G. Lewis, and C. Kästner, "Data leakage in notebooks: Static detection and better processes," in *Proc. 37th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2022, pp. 1–12.

[4] N. Alturayeif and J. Hassine, "Data leakage detection in machine learning code: transfer learning, active learning, or low-shot prompting?" *PeerJ Computer Science*, vol. 11, p. e2730, 2025.

[5] S. Grafberger, S. Guha, J. Stoyanovich, and S. Schelter, "MLINSPECT: A data distribution debugger for machine learning pipelines," in *Proc. 2021 Int. Conf. on Management of Data (SIGMOD)*, 2021, pp. 2736–2739.

[6] DataRobot, "Data quality checks: target leakage," DataRobot documentation, 2026.

[7] H2O.ai, "Data leakage and shift detection in Driverless AI," Driverless AI 1.11 User Guide, 2026.

[8] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: Towards automating data science endeavors," in *IEEE Int. Conf. on Data Science and Advanced Analytics (DSAA)*, 2015, pp. 1–10.

[9] J. Robinson, R. Ranjan, W. Hu, K. Huang, J. Han, A. Dobles, M. Fey, J. E. Lenssen, Y. Yuan, Z. Zhang *et al.*, "RelBench: A benchmark for deep learning on relational databases," *Advances in Neural Information Processing Systems*, vol. 37, pp. 21 330–21 341, 2024.

[10] P. Marian, "PITBench: Relational databases prepared for point-in-time checking across the ML stack," https://github.com/PavelMarian/PITBench, 2026.

[11] Fonseca *et al.*, "Look-ahead-freedom for backtesting code," arXiv:2607.04958, 2026.